

ORACLE SQL DEEP DIVE

Why ROWNUM = 1 Works But ROWNUM = 2 Does Not

A detailed explanation of Oracle's row-numbering semantics, query-block behavior, stopkey optimization, ordering, and pagination

The essential answer

ROWNUM is not a permanent position attached to a table row. In a query block, Oracle starts with a current value of 1 and tests candidate rows. The value advances only after a row is accepted and returned by that query block. Therefore ROWNUM = 1 can accept the first candidate and advance the counter, but ROWNUM = 2 rejects every candidate while the current value remains 1 forever.

Audience: Oracle SQL learners, developers, and database administrators

Prepared as a practical, runnable reference

Contents

1	The direct explanation
2	What ROWNUM actually represents
3	The internal acceptance-counter mechanism
4	Step-by-step traces for = 1, = 2, and = 3
5	Why <= N works and >= N does not
6	ROWNUM and ORDER BY
7	Correct ways to retrieve the nth row
8	Correct top-N and pagination patterns
9	Optimizer behavior and STOPKEY
10	Runnable HR schema examples
11	Common misconceptions and final rules

How to read the examples

The examples use the HR.EMPLOYEES table where convenient. The exact first physical row is intentionally not assumed unless an ORDER BY is placed inside an inline view. Without an explicit ordering rule, SQL does not promise which qualifying row is encountered first.

1. The direct explanation

The query **WHERE ROWNUM = 1** succeeds because the first candidate row is evaluated with the current ROWNUM value 1. The predicate is true, so the row is returned.

The query **WHERE ROWNUM = 2** fails because the first candidate row is also evaluated with current ROWNUM 1. The predicate is false. Because no row was accepted, the counter does not advance. The next candidate is again tested with 1, then the next, and so on. No candidate can ever see 2.

```
-- Returns one row
SELECT employee_id, last_name
FROM employees
WHERE ROWNUM = 1;

-- Returns no rows
SELECT employee_id, last_name
FROM employees
WHERE ROWNUM = 2;
```

A useful mental model

ROWNUM behaves like an output-row counter for the current query block. It is closer to "the number this row would receive if it is accepted" than to "the row's stored position in the table."

2. What ROWNUM actually represents

ROWNUM is a pseudocolumn. It looks like a column in a SELECT statement, but its values are not stored in the table. Oracle exposes it while producing rows from a query block. The first accepted row has value 1, the second accepted row has value 2, and so on. [1]

- It is not a primary key, ROWID, or physical row position.
- The same stored row can receive different ROWNUM values in different executions or query blocks.
- The access path can affect which row is encountered first when no deterministic ORDER BY is applied before numbering.
- Every query block has its own ROWNUM scope. An inline view can assign rows in one block and filter them in an outer block.

A normal limiting use

```
SELECT employee_id,
       last_name,
       ROWNUM AS rn
FROM employees
WHERE department_id = 50
AND ROWNUM <= 5;
```

This query can return up to five employees from department 50. It does not mean those employees permanently own row numbers 1 through 5.

3. The internal acceptance-counter mechanism

The following pseudocode is a conceptual model of the documented behavior. It is not literal Oracle source code, and the optimizer is free to choose different physical operators, but it accurately explains the semantics. Oracle Ask TOM describes the key rule: ROWNUM advances only after a row successfully passes the predicate and is output. [2]

Conceptual algorithm

```
current_rownum := 1;

FOR each candidate_row produced by the row source LOOP
  IF predicate_is_true(candidate_row, current_rownum) THEN
    output candidate_row with ROWNUM = current_rownum;
    current_rownum := current_rownum + 1;
  END IF;
END LOOP;
```



The decisive detail

A failed row does not consume a ROWNUM value. Therefore there cannot be an accepted row numbered 2 until an accepted row numbered 1 already exists in the same query block.

4. Step-by-step traces

4.1 Trace: WHERE ROWNUM = 1

Candidate	Current ROWNUM	Predicate	Pass?	Output	Next ROWNUM
Row A	1	1 = 1	Yes	Row A (rn=1)	2
Row B	2	2 = 1	No	-	2
Row C	2	2 = 1	No	-	2

The first candidate passes. If the query has only the condition ROWNUM = 1, later candidates are rejected because the current value is now 2. The result is exactly one row when at least one candidate exists.

4.2 Trace: WHERE ROWNUM = 2

Candidate	Current ROWNUM	Predicate	Pass?	Output	Next ROWNUM
Row A	1	1 = 2	No	-	1
Row B	1	1 = 2	No	-	1
Row C	1	1 = 2	No	-	1
Row D	1	1 = 2	No	-	1

Every candidate is tested with 1 because none passes. The counter is trapped at 1. The result is zero rows.

4.3 Trace: WHERE ROWNUM = 3

Candidate	Current ROWNUM	Predicate	Pass?	Output	Next ROWNUM
Row A	1	1 = 3	No	-	1
Row B	1	1 = 3	No	-	1
Row C	1	1 = 3	No	-	1

The same reasoning applies to 3, 4, or any value greater than 1. A row numbered 3 would require rows 1 and 2 to have been accepted first, but the equality predicate rejects row 1 immediately.

General rule for equality

In a single query block, WHERE ROWNUM = n can directly return rows only when n = 1. For n > 1, the predicate prevents the sequence from ever reaching n.

5. Why <= N works and >= N does not

5.1 Upper bounds allow the counter to grow

```
SELECT employee_id, last_name, ROWNUM AS rn
FROM employees
WHERE ROWNUM <= 3;
```

Candidate	Current ROWNUM	Predicate	Pass?	Output	Next ROWNUM
Row A	1	1 <= 3	Yes	rn=1	2
Row B	2	2 <= 3	Yes	rn=2	3
Row C	3	3 <= 3	Yes	rn=3	4
Row D	4	4 <= 3	No	-	4

Values 1, 2, and 3 all satisfy the condition, so each accepted row increments the counter. Oracle can stop once the upper bound cannot be satisfied.

5.2 Lower bounds block the first required row

```
-- Both return no rows in a single query block
SELECT * FROM employees WHERE ROWNUM >= 2;
SELECT * FROM employees WHERE ROWNUM > 1;
```

The first candidate is evaluated with value 1. Since 1 is not at least 2, it is rejected and the counter remains 1. Oracle documentation explicitly notes that ROWNUM conditions greater than a positive integer are always false in this form. [1]

5.3 Predicate summary

Predicate in one query block	Typical result	Reason
ROWNUM = 1	0 or 1 row	First candidate may pass.
ROWNUM = 2	0 rows	First candidate fails; counter stays 1.
ROWNUM <= 5	Up to 5 rows	Values 1 through 5 can pass sequentially.
ROWNUM < 5	Up to 4 rows	Values 1 through 4 can pass sequentially.
ROWNUM > 1	0 rows	Value 1 must pass before 2 can exist, but it does not.

ROWNUM BETWEEN 2 AND 5	0 rows	Lower bound rejects the first required row.
------------------------	--------	---

6. ROWNUM and ORDER BY

In the same query block, ROWNUM limiting is applied to rows as they are produced, and a later ORDER BY sorts the rows that survived. It does not first sort the entire result and then assign ROWNUM. Oracle documentation warns that access paths can change which rows are selected. [1]

Incorrect top-N pattern

```
-- Usually not what "top 5 salaries" means
SELECT employee_id, last_name, salary
FROM employees
WHERE ROWNUM <= 5
ORDER BY salary DESC;
```

This takes whichever five rows are encountered first, then sorts only those five by salary. It does not guarantee the five highest salaries in the table.

Correct top-N pattern

```
-- Correct: sort in an inner query block, limit outside
SELECT employee_id, last_name, salary
FROM (
  SELECT employee_id, last_name, salary
  FROM employees
  ORDER BY salary DESC, employee_id
)
WHERE ROWNUM <= 5;
```

The inner query block establishes the order. The outer block then numbers and limits the already ordered stream. Adding employee_id as a tie-breaker makes the ordering deterministic.

Query-block principle

To make ROWNUM operate after an ORDER BY, place the ORDER BY in an inner query block and apply ROWNUM in an outer query block.

7. Correct ways to retrieve the nth row

7.1 Preferred: ROW_NUMBER()

ROW_NUMBER() is an analytic function that assigns unique sequential numbers according to an explicit ORDER BY. Oracle documents it as the more capable choice for ordered row numbering. [3]

```
SELECT employee_id, last_name, salary
FROM (
  SELECT e.employee_id,
         e.last_name,
         e.salary,
         ROW_NUMBER() OVER (
           ORDER BY e.salary DESC, e.employee_id
         ) AS rn
  FROM employees e
)
WHERE rn = 2;
```

This query correctly returns the second row in the stated salary order. The analytic row number is first computed in the inner query, then filtered in the outer query.

7.2 Modern syntax: OFFSET ... FETCH

```
SELECT employee_id, last_name, salary
FROM employees
ORDER BY salary DESC, employee_id
OFFSET 1 ROW
FETCH NEXT 1 ROW ONLY;
```

OFFSET skips the first ordered row, and FETCH returns the next one. Oracle recommends an ORDER BY for consistent results when using the row-limiting clause. [4]

7.3 Legacy ROWNUM nesting

```
SELECT employee_id, last_name, salary
FROM (
    SELECT ordered_rows.*,
           ROWNUM AS rn
    FROM (
        SELECT employee_id, last_name, salary
        FROM employees
        ORDER BY salary DESC, employee_id
    ) ordered_rows
    WHERE ROWNUM <= 2
)
WHERE rn = 2;
```

The inner condition ROWNUM <= 2 permits row numbers 1 and 2 to be created. The outer condition can then select rn = 2 because rn is now a normal projected alias from the inner result.

8. Correct top-N and pagination patterns

8.1 First N ordered rows

```
-- Modern form
SELECT employee_id, last_name, salary
FROM employees
ORDER BY salary DESC, employee_id
FETCH FIRST 10 ROWS ONLY;
```

8.2 Rows 6 through 10 with ROW_NUMBER()

```
SELECT employee_id, last_name, salary
FROM (
    SELECT e.*,
           ROW_NUMBER() OVER (
               ORDER BY salary DESC, employee_id
           ) AS rn
    FROM employees e
)
WHERE rn BETWEEN 6 AND 10
ORDER BY rn;
```

8.3 Rows 6 through 10 with legacy ROWNUM

```
SELECT employee_id, last_name, salary, rn
FROM (
  SELECT q.*, ROWNUM AS rn
  FROM (
    SELECT employee_id, last_name, salary
    FROM employees
    ORDER BY salary DESC, employee_id
  ) q
  WHERE ROWNUM <= 10
)
WHERE rn >= 6
ORDER BY rn;
```

The upper bound must be applied in the inner query block so that values 1 through 10 can be generated. The lower bound is applied outside, after rn has become an ordinary column in the inline view result.

Why the two-level pagination pattern works

Inner block: generate and cap the sequence with ROWNUM <= maximum_row. Outer block: discard earlier rows using the saved alias rn >= minimum_row.

9. Optimizer behavior and STOPKEY

ROWNUM predicates are not merely cosmetic. An upper-bound condition such as ROWNUM <= N often allows the optimizer to stop the row source after N accepted rows. Execution plans commonly show a STOPKEY-related operator, such as COUNT STOPKEY, although the exact plan depends on the Oracle release, indexes, statistics, joins, and transformations.

Inspect the plan in your environment

```
EXPLAIN PLAN FOR
SELECT employee_id, last_name
FROM employees
WHERE department_id = 50
AND ROWNUM <= 1;

SELECT *
FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Typical conceptual plan shape - exact output may vary

```
SELECT STATEMENT
  COUNT STOPKEY
    TABLE ACCESS ... EMPLOYEES
```

- STOPKEY means the engine has a row-count stopping condition.
- For ROWNUM <= 1, Oracle can stop as soon as the first qualifying row is accepted.
- For ordered top-N queries, the optimizer can use specialized top-N processing rather than always sorting every row completely.
- The semantic explanation remains the acceptance-counter rule even when the optimizer implements it with transformed operators.

Semantics versus implementation

The counter model explains what the SQL means. The execution plan shows how Oracle chose to implement that meaning for one statement in one environment. Do not assume every release or access path will display the same operator names.

10. Runnable HR schema examples

Basic behavior

```
-- 1) One arbitrary qualifying row
SELECT employee_id, last_name, ROWNUM AS rn
FROM employees
WHERE department_id = 50
  AND ROWNUM = 1;

-- 2) No rows: the sequence never reaches 2
SELECT employee_id, last_name, ROWNUM AS rn
FROM employees
WHERE department_id = 50
  AND ROWNUM = 2;

-- 3) Up to three qualifying rows
SELECT employee_id, last_name, ROWNUM AS rn
FROM employees
WHERE department_id = 50
  AND ROWNUM <= 3;
```

Correct nth-row query

```
-- The second employee by employee_id
SELECT employee_id, last_name
FROM (
  SELECT e.employee_id,
         e.last_name,
         ROW_NUMBER() OVER (
           ORDER BY e.employee_id
         ) AS rn
  FROM employees e
)
WHERE rn = 2;
```

Preferred ordered top-N query

```
-- Five highest salaries, deterministic for ties
SELECT employee_id, last_name, salary
FROM employees
ORDER BY salary DESC, employee_id
FETCH FIRST 5 ROWS ONLY;
```

Modern pagination

```
-- Page 3 with page size 10: rows 21-30
SELECT employee_id, last_name, salary
FROM employees
ORDER BY employee_id
OFFSET 20 ROWS
FETCH NEXT 10 ROWS ONLY;
```

11. Common misconceptions and final rules

Misconception	Correction
"ROWNUM is assigned to every table row before WHERE."	No. It is generated as rows are accepted in a query block.
"ROWNUM = 2 means skip row 1 and return row 2."	No. The equality rejects the first candidate, so row number 2 is never created.
"ROWNUM <= 5 ORDER BY salary DESC gives the five highest salaries."	Not in the same query block. It limits first and sorts the survivors.
"A row always has the same ROWNUM."	No. ROWNUM depends on the query block and row-production order.
"ROWNUM and ROW_NUMBER() are interchangeable."	No. ROW_NUMBER() supports explicit ordered numbering and is filterable through an outer query.

Final rules to remember

1. ROWNUM starts at 1 for the first accepted row in a query block.
2. The counter advances only after a row passes and is output.
3. ROWNUM = 1 can work; ROWNUM = n for n > 1 cannot directly work in the same query block.
4. Upper bounds such as ROWNUM <= N work because they allow the sequence to grow from 1.
5. Use an inline view when ordering must occur before ROWNUM limiting.
6. Use ROW_NUMBER() or OFFSET/FETCH for clear ordered nth-row and pagination queries.
7. Always include a deterministic ORDER BY when the identity of the selected row matters.

One-sentence memory aid

ROWNUM does not count candidates that fail; it counts rows that successfully leave the current query block.

Sources and further reading

[\[1\] Oracle Database 26 SQL Language Reference - ROWNUM Pseudocolumn](#)

Defines ROWNUM, warns about ORDER BY in the same query, and explains that conditions requiring values greater than 1 cannot succeed in the direct form.

[\[2\] Oracle Ask TOM - On ROWNUM and Limiting Results](#)

Explains the acceptance-and-increment mechanism and gives conceptual pseudocode for limiting queries.

[\[3\] Oracle Database 26 SQL Language Reference - ROW NUMBER](#)

Documents ordered analytic row numbering.

[\[4\] Oracle Database 26 SQL Language Reference - SELECT, row limiting clause](#)

Documents OFFSET and FETCH and recommends ORDER BY for deterministic results.

Technical note: The procedural traces in this guide describe SQL semantics. Exact physical execution plans can vary because of optimizer transformations, database version, indexes, statistics, parallel execution, and the surrounding query.